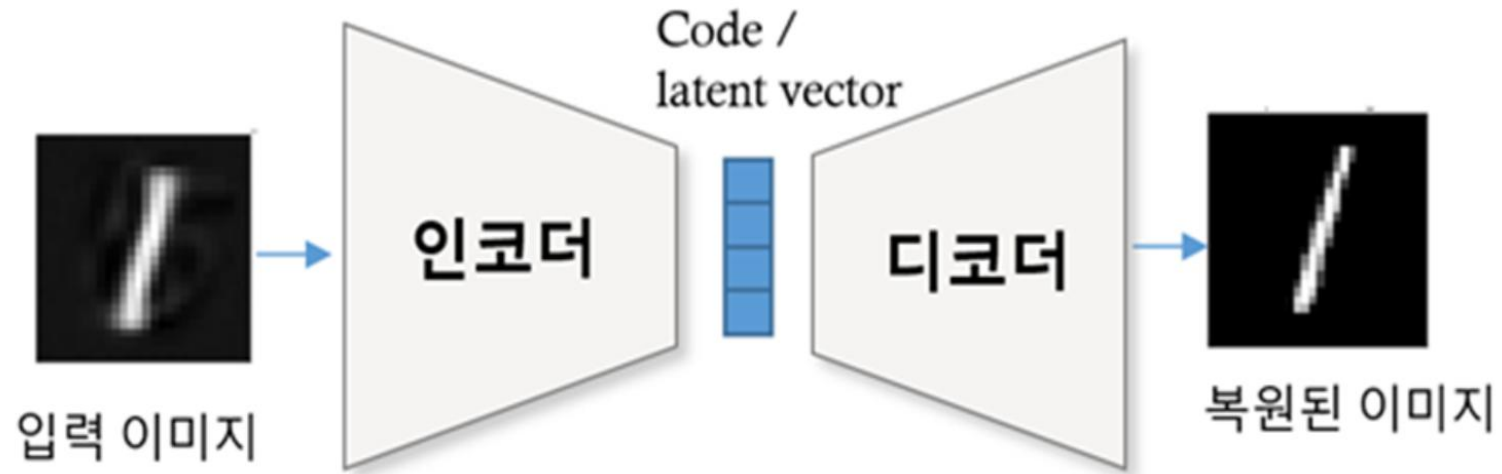
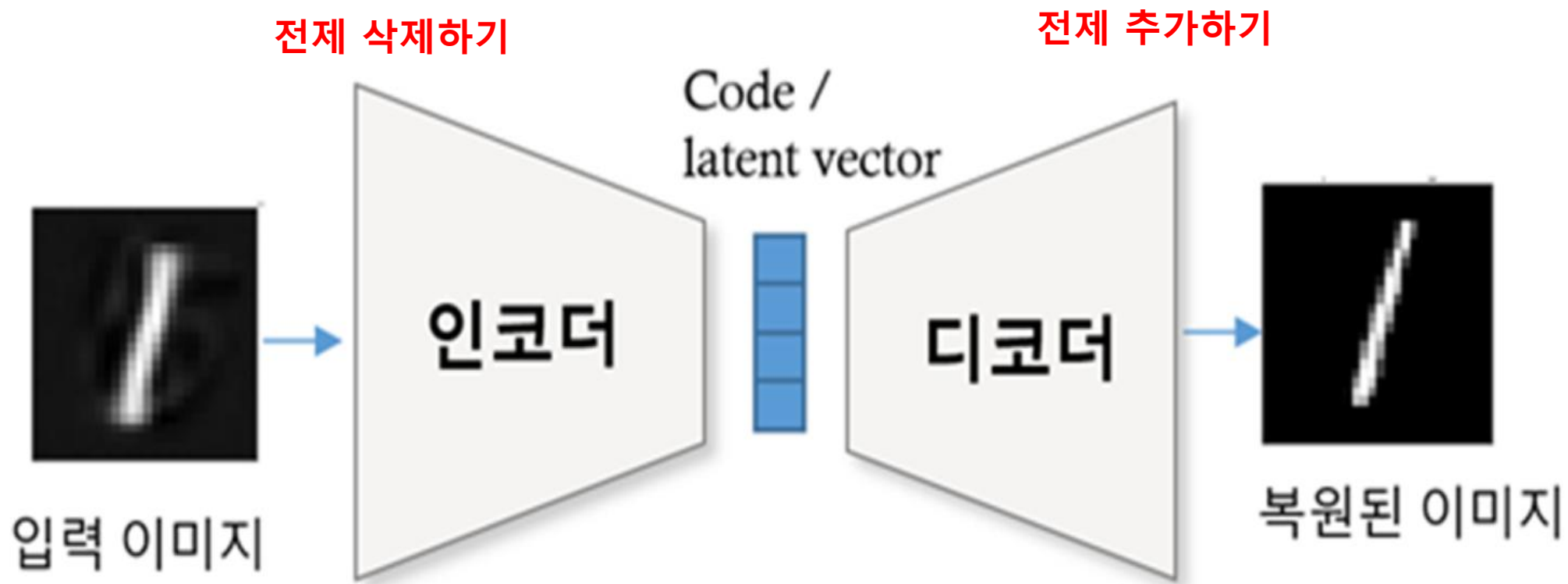


IT := Information Technology





기억 : 저장하고 끄집어 내기

<> PYTHON CODE

1. 리스트 생성 및 저장 (끝에 추가)

```
my_list = ['사과', '바나나']  
my_list.append('체리')
```

2. 특정 위치에 저장 (삽입)

```
my_list.insert(1, '오렌지')
```

3. 데이터 꺼내기 (인덱스 사용)

```
first_item = my_list[0] # '사과'
```

4. 데이터 꺼내고 삭제하기

```
last_item = my_list.pop() # '체리'
```

👁 VISUALIZATION

리스트는 데이터가 **순서대로** 기차 객실처럼 연결되어 저장됩니다. 각 칸은 **인덱스(0, 1, 2...)** 번호를 가집니다.



→ 데이터가 뒤로 계속 이어집니다 (append)

기억 : 저장하고 끄집어 내기

<> PYTHON CODE

```
# 1. 딕셔너리 생성
my_dict = {}

# 2. 데이터 저장 (Key: Value)
my_dict['이름'] = '홍길동'
my_dict['나이'] = 25

# 3. 데이터 꺼내기 (Key 사용)
name = my_dict['이름'] # '홍길동'

# 4. 안전하게 꺼내기 (get 사용)
job = my_dict.get('직업', '무직')
# Key가 없으면 기본값 반환
```

👁 VISUALIZATION

딕셔너리는 사전처럼 단어 (Key) 를 찾으면 그 뜻 (Value) 이 나오는 **열쇠와 자물쇠** 쌍 구조입니다. 순서가 중요하지 않습니다.

열쇠 (Key)

값 (Value)

'이름'



'홍길동'

'나이'



25

'직업'



?? (get fallback)

기억 : 저장하고 끄집어 내기

<> PYTHON CODE

```
# 1. 셋(집합) 생성
my_set = set()

# 2. 데이터 저장 (순서 없음, 중복 불가)
my_set.add('고양이')
my_set.add('강아지')
my_set.add('고양이') # 무시됨 (중복)

# 3. 데이터 꺼내기 (인덱스 불가!)
# 임의의 요소를 꺼내고 제거할 때
item = my_set.pop()

# 4. 데이터 확인 (꺼내지 않고 존재 여부만)
if '강아지' in my_set:
    print("있습니다!")
```

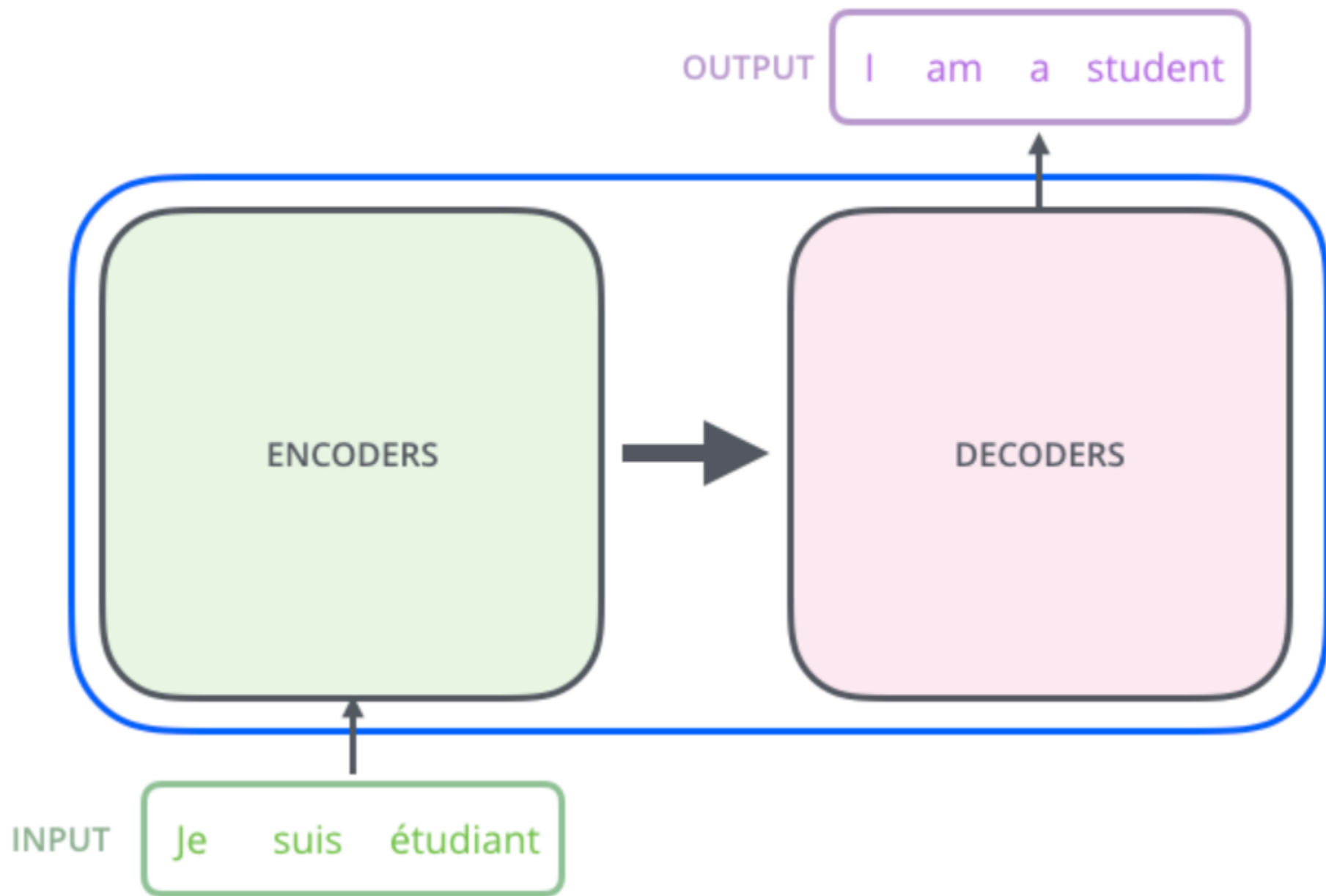
👁 VISUALIZATION

셋은 마법의 주머니와 같습니다. 순서가 섞여 들어가며, **똑같은 데이터는 절대 2개가 될 수 없습니다** (중복 제거).



요소를 꺼내려면 pop()을 쓰지만, 무엇이 나올지 알 수 없습니다 (임의 추출).





Relational Algebra

Customers					
customer_id	first_name	last_name	age	country	
1	John	Doe	31	USA	
2	Robert	Luna	22	USA	
3	David	Robinson	22	UK	
4	John	Reinhardt	25	UK	
5	Betty	Doe	28	UAE	

Relational Algebra

Customers				
customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Key

Value

Relational Algebra

Query

```
SELECT age, country
FROM Customers
WHERE first_name = 'John'
```

Customers				
customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Key

Value

Relational Algebra

```
SELECT age, country
FROM Customers
WHERE first_name = 'John'
```

Query

Customers				
customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Relational Algebra

```
SELECT age, country
FROM Customers
WHERE first_name = 'John'
```

Query



Key

Value

Customers				
customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

age	country
31	USA
25	UK

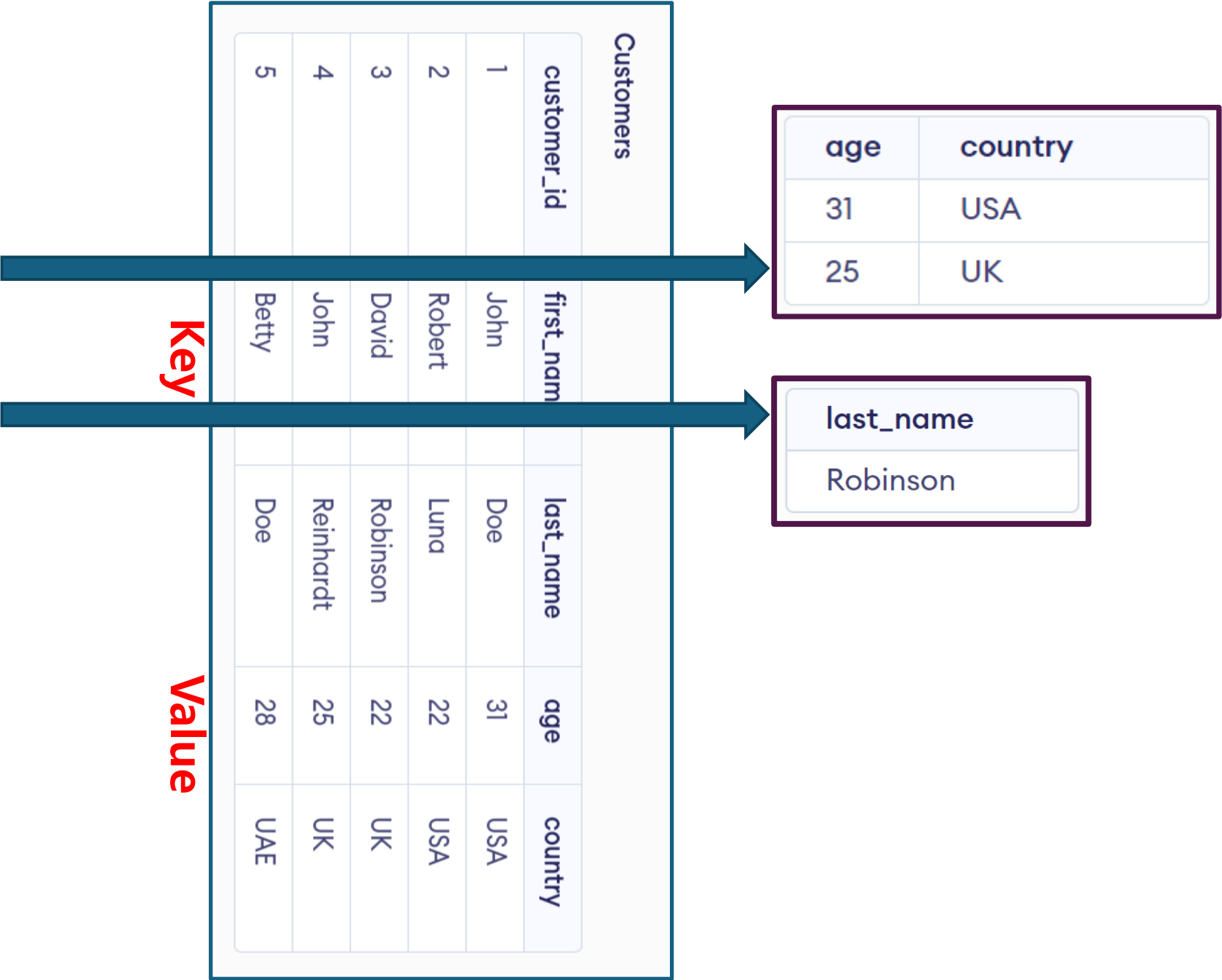
Relational Algebra

Query

```
SELECT age, country
FROM Customers
WHERE first_name = 'John'
```

Query

```
SELECT last_name
FROM Customers
WHERE first_name = 'David'
```



Relational Algebra

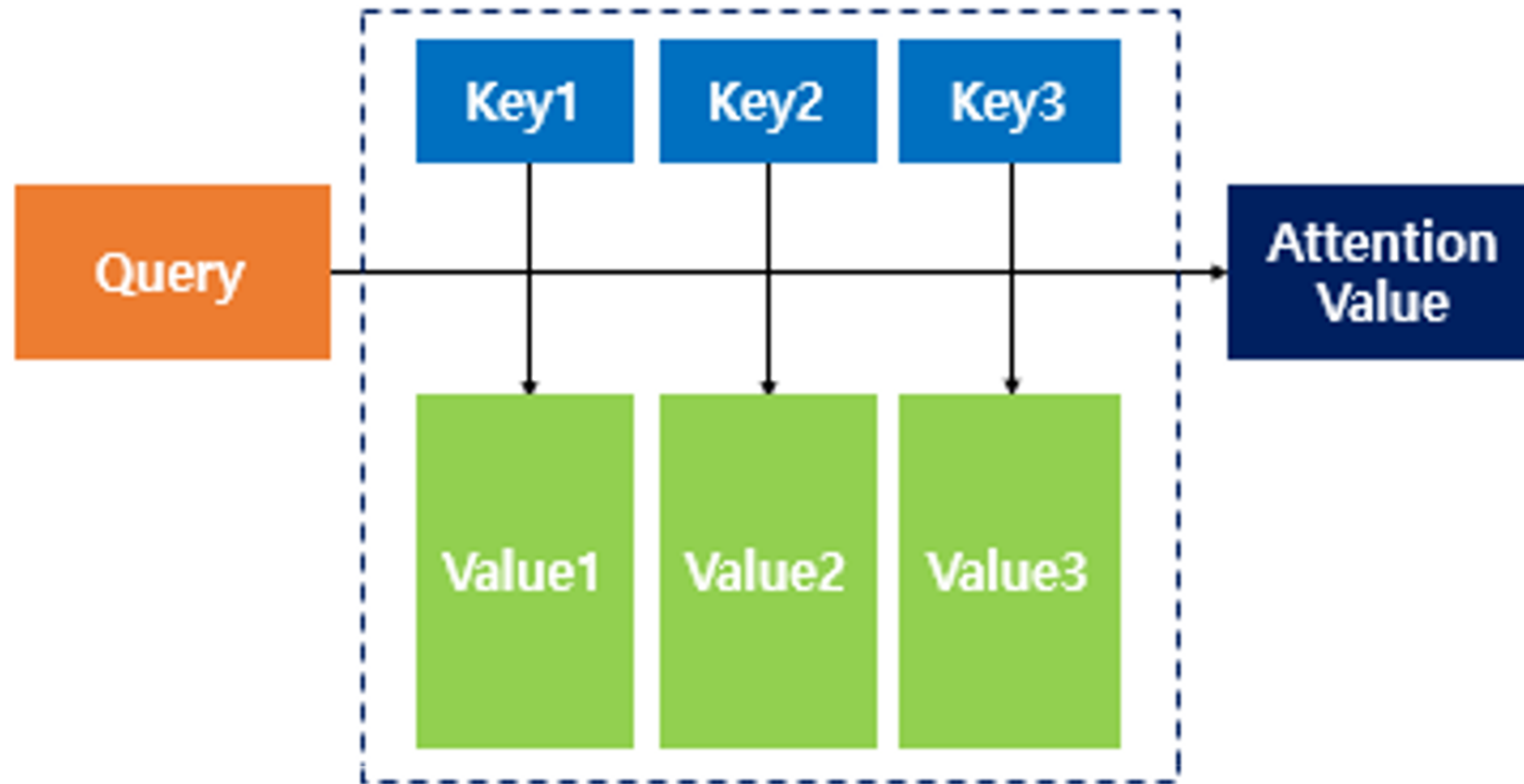
first_name	last_name	age	country
John	Doe	31	USA
Robert	Luna	22	USA
David	Robinson	22	UK
John	Reinhardt	25	UK
Betty	Doe	28	UAE

```
SELECT age, country  
WHERE first_name = 'John'
```

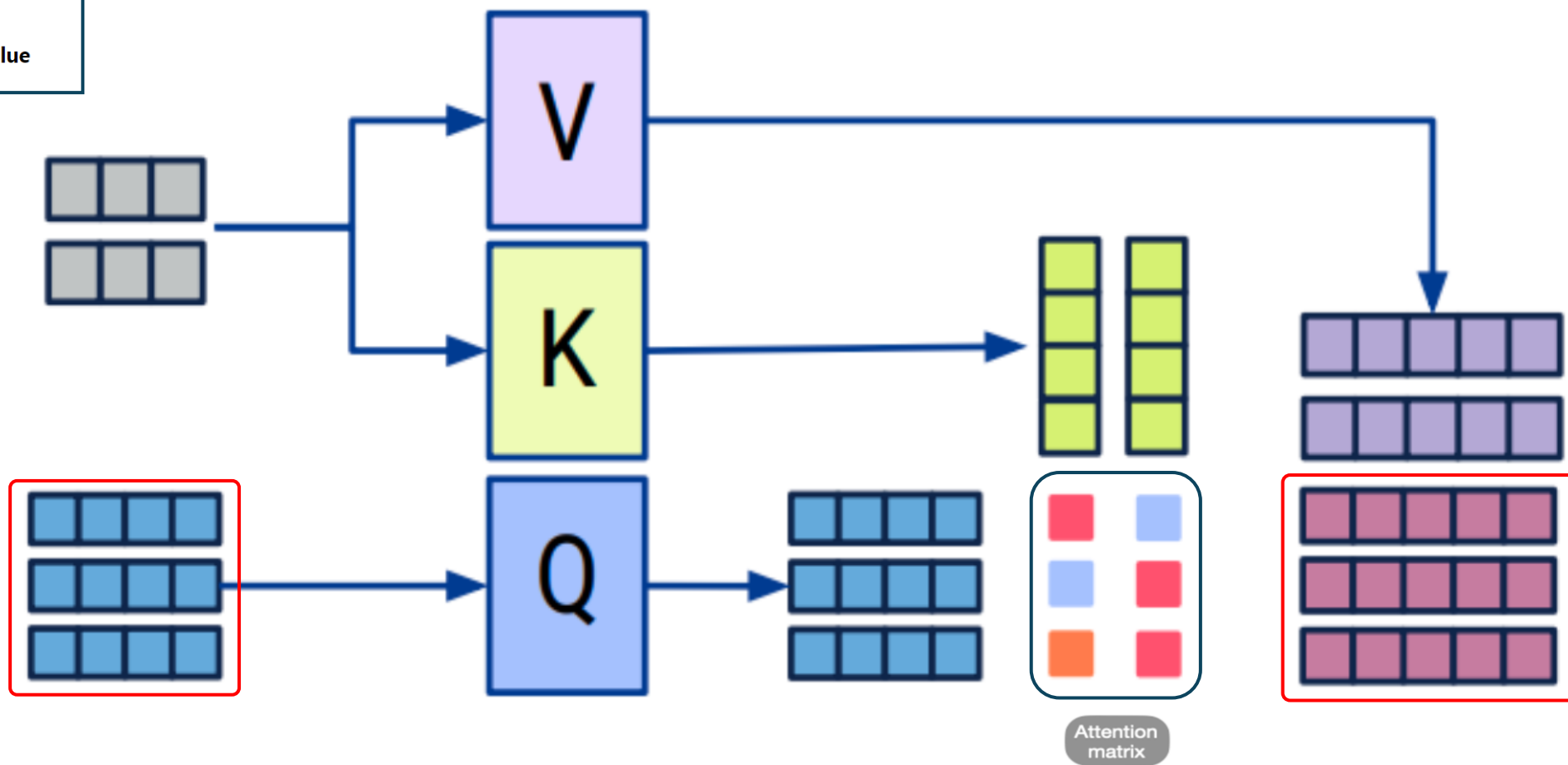
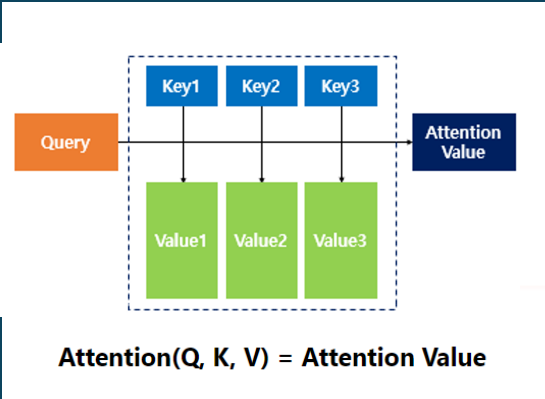
age	country
31	USA
25	UK

```
SELECT country  
WHERE age=25
```

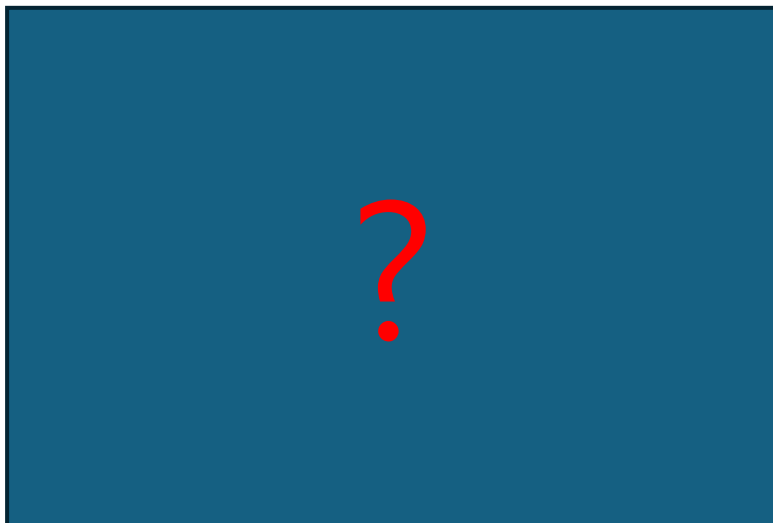
country
UK



$\text{Attention}(Q, K, V) = \text{Attention Value}$



$(3, 4, 5)$

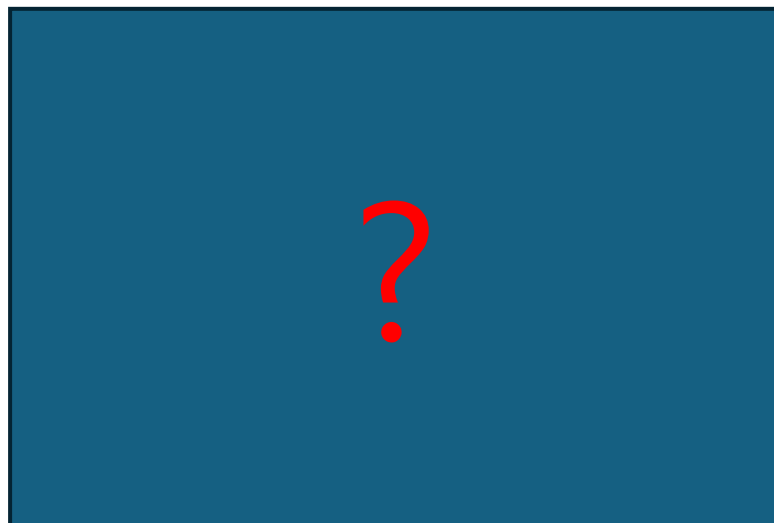


$(0.8, 7.2, -9, 12)$

$(3, 4, 5)$



$(0.5, -8, 1)$



$(0.8, 7.2, -9, 12)$



$(4, -2, 3.3, 5.4)$

